

The Bounded Pathwidth of Control-flow Graphs

GIOVANNA KOBUS CONRADO, AMIR KAFSHDAR GOHARSHADY, and CHUN KIT LAM

Hong Kong University of Science and Technology, Hong Kong
OOPSLA 23



香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

program

```
program
  while1 B do
    if1 B then
      break1
    else
      α1
  fi1;
  while2 B do
    if2 B then
      break2
    else
      α2;
      continue2
    fi2
  done2
done1;
if3 B then
  α3
else
  α4
fi3
margorp
```

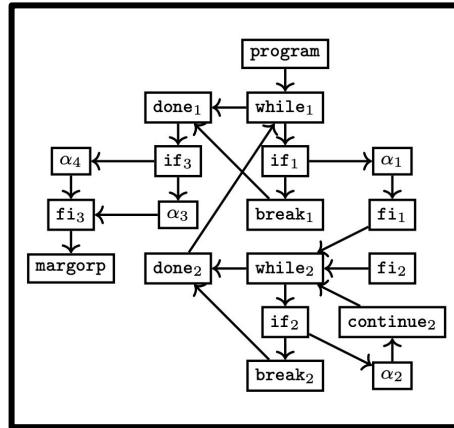
program

```

program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
  while2 B do
    if2 B then
      break2
    else
      α2;
      continue2
    fi2
  done2
done1;
if3 B then
  α3
else
  α4
fi3
margorp
  
```



control-flow graph

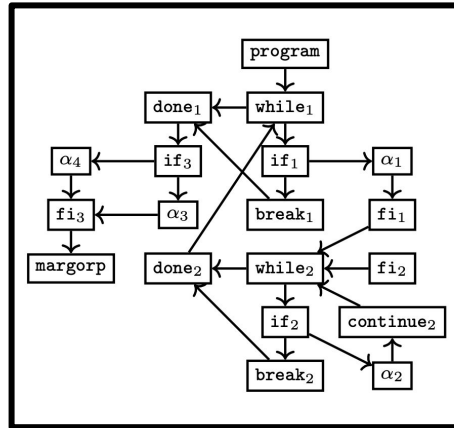


program

```
program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
  while2 B do
    if2 B then
      break2
    else
      α2;
      continue2
    fi2
  done2
done1;
if3 B then
  α3
else
  α4
fi3
margorp
```

compiler
problem

control-flow graph

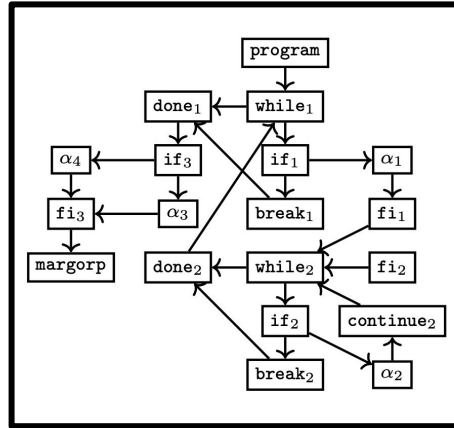


graph problem

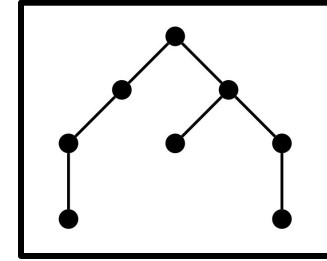
program

```
program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
    while2 B do
      if2 B then
        break2
      else
        α2;
        continue2
      fi2
    done2
  done1;
  if3 B then
    α3
  else
    α4
  fi3
  margorp
```

control-flow graph



tree decomposition



Currently used in:

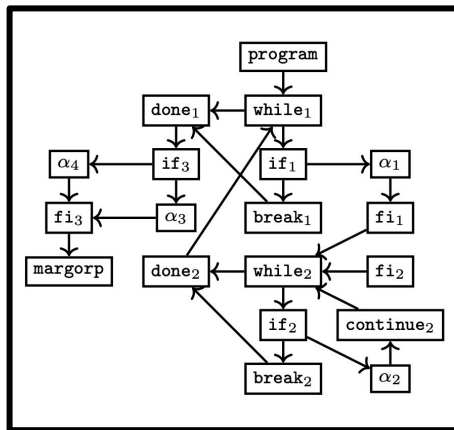
- Register allocation
- Redundancy elimination
- Reducing cache misses
- Data-flow analysis
- ...

program

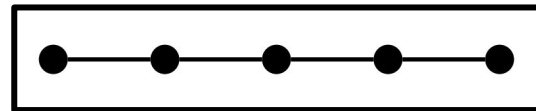
```

program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
    while2 B do
      if2 B then
        break2
      else
        α2;
        continue2
      fi2
    done2
  done1;
  if3 B then
    α3
  else
    α4
  fi3
  margorp
  
```

control-flow graph



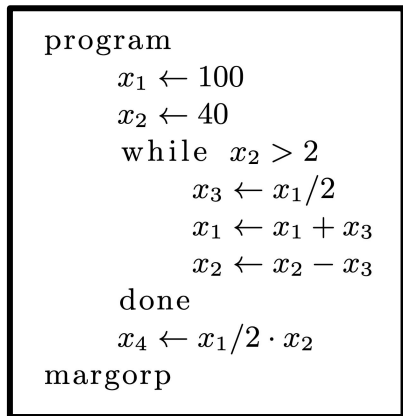
our work path decomposition



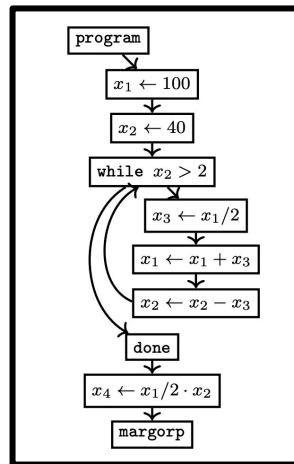
- Can solve all the same problems
- Easier algorithms
- Large speedups

Register allocation

Is it possible to map the variables in a program to r registers without interference?



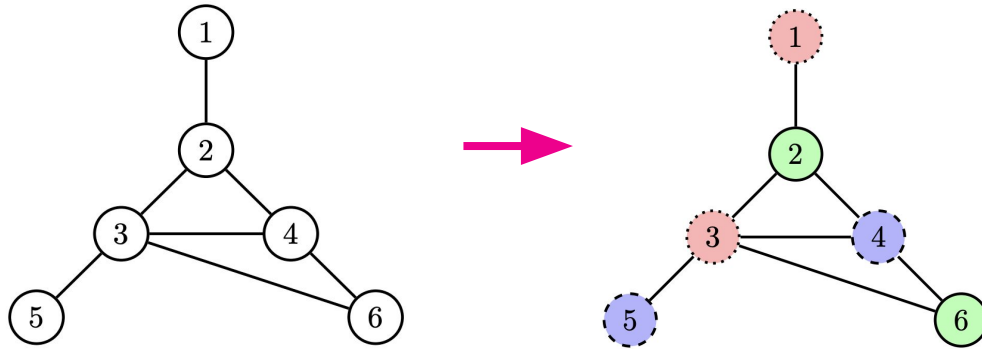
register allocation



graph coloring

Graph coloring

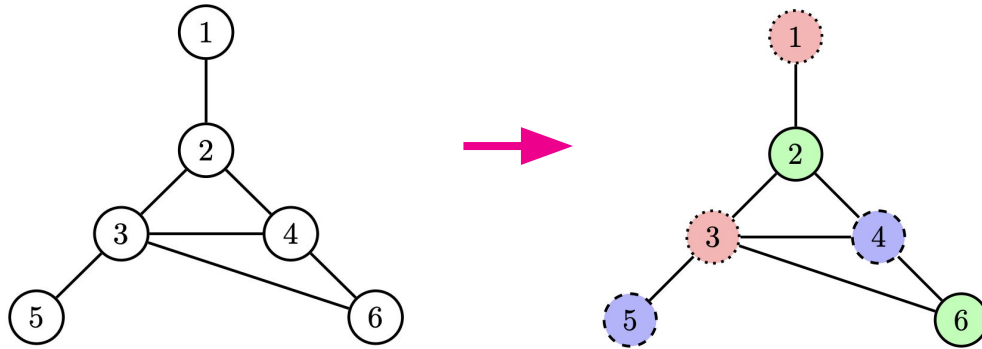
Can a graph be colored with r colors in such a way that adjacent vertices have different colors?



3 colors $\rightarrow$ NP-complete $\rightarrow O(n3^n)$

Graph coloring

Can a graph be colored with r colors in such a way that adjacent vertices have different colors?



Not fast enough

How do we solve these problems in control-flow graphs?

- Prove that control-flow graphs are structured
- Solve hard graph problems efficiently for structured graphs

Control-flow graph

Graph representation of a program

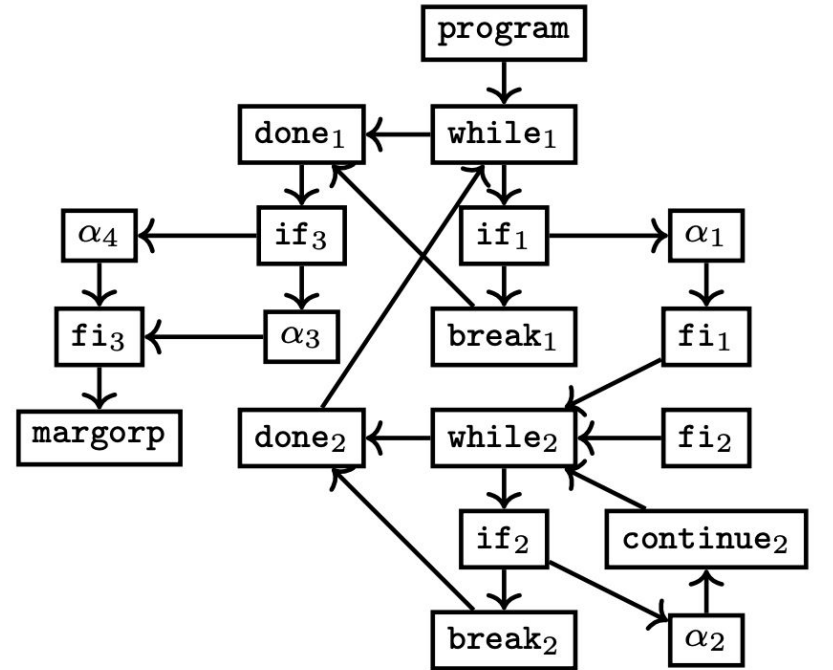
Used in **intraprocedural** program analysis

```

program
  while1 B do
    if1 B then
      break1
    else
      α1
      fi1;
  while2 B do
    if2 B then
      break2
    else
      α2;
      continue2
    fi2
  done2
done1;
if3 B then
  α3
else
  α4
fi3
margorp

```

line $\longrightarrow$ vertex
 execution $\longrightarrow$ path



Nesting depth

Maximum “nestedness” of a statement in the program

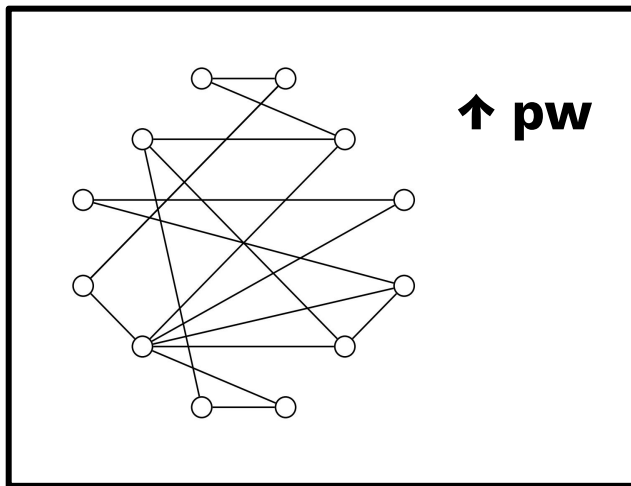
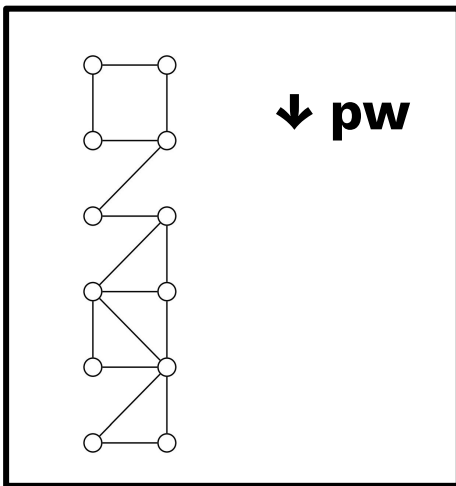
Most coding standards discourage or forbid nesting depth of more than 5

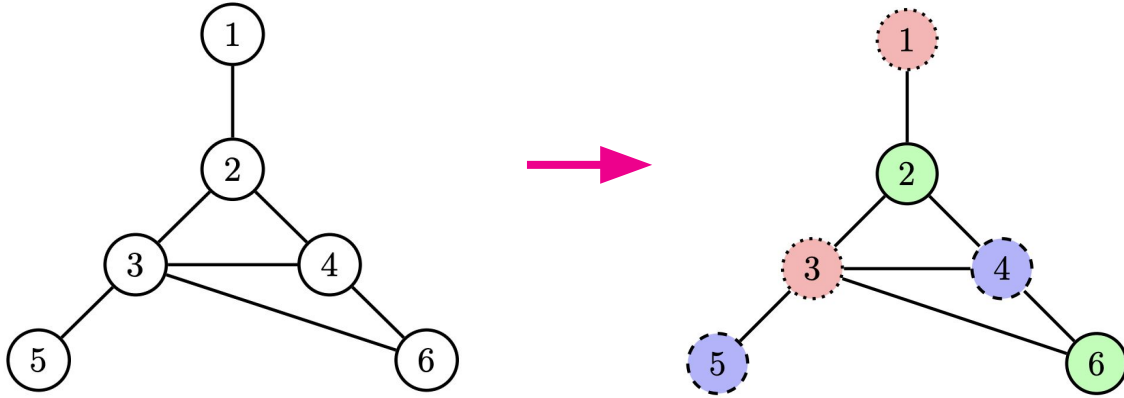
```
program
  while1 B do
    if1 B then
      break1
    else
       $\alpha_1$ 
    fi1;
  while2 B do
    if2 B then
      break2
    else
       $\alpha_2$ ;
      continue2
    fi2
  done2
done1;
margorp
```

How to solve graph coloring faster?

Pathwidth: measure of how complicated a graph is

The bigger the number, the more complicated the graph





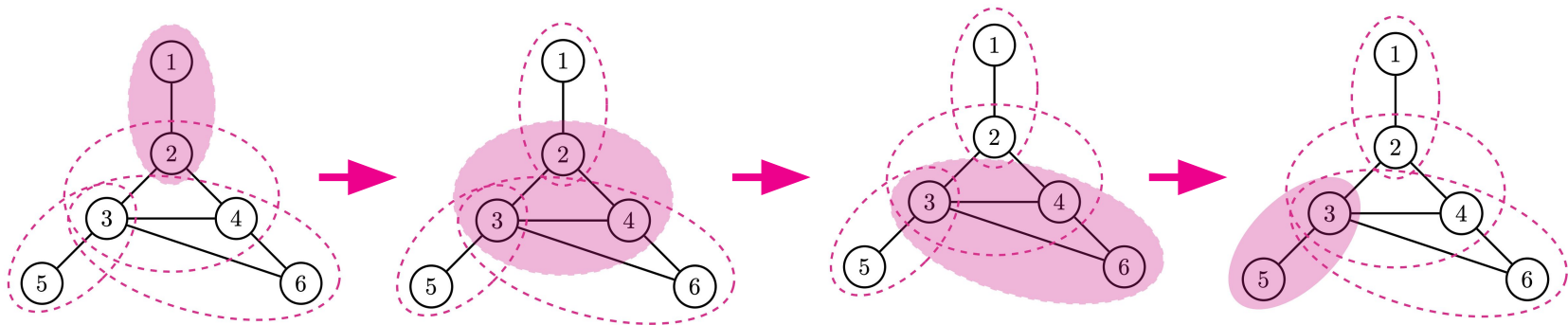
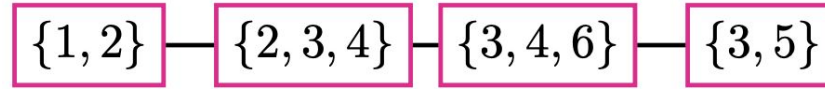
$$O(n^{3^{pw}})$$

We can solve graph coloring fast for path-like graphs

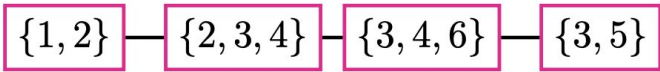
Pathwidth

How do we measure a graph's similarity to a path?

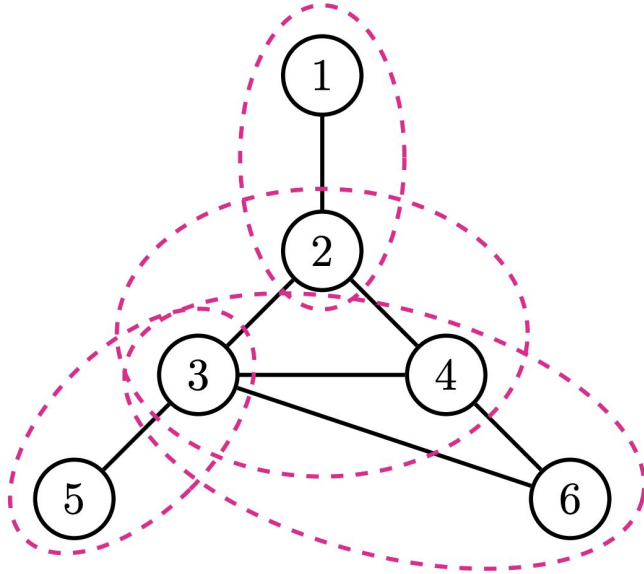
Path decomposition

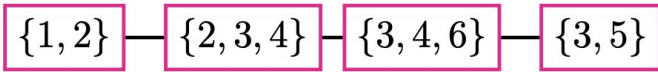


- All edges are visited
- We never revisit a vertex

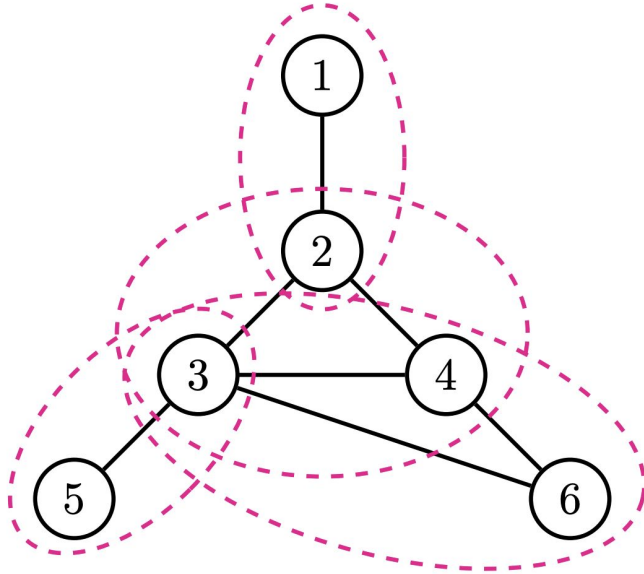


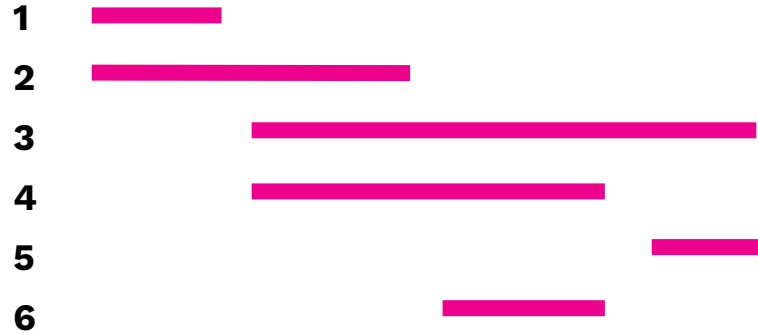
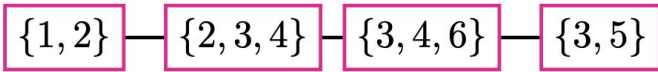
Sequence of bags of vertices





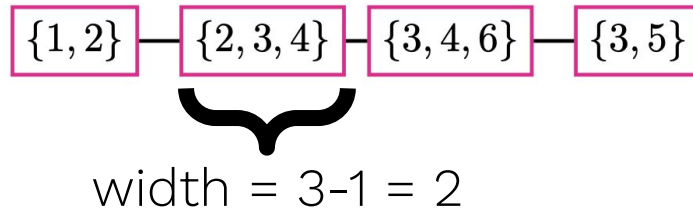
**All edges must be
contained in some bag**





The bags containing any particular vertex are contiguous

The width of a path decomposition is the size of its largest bag minus one



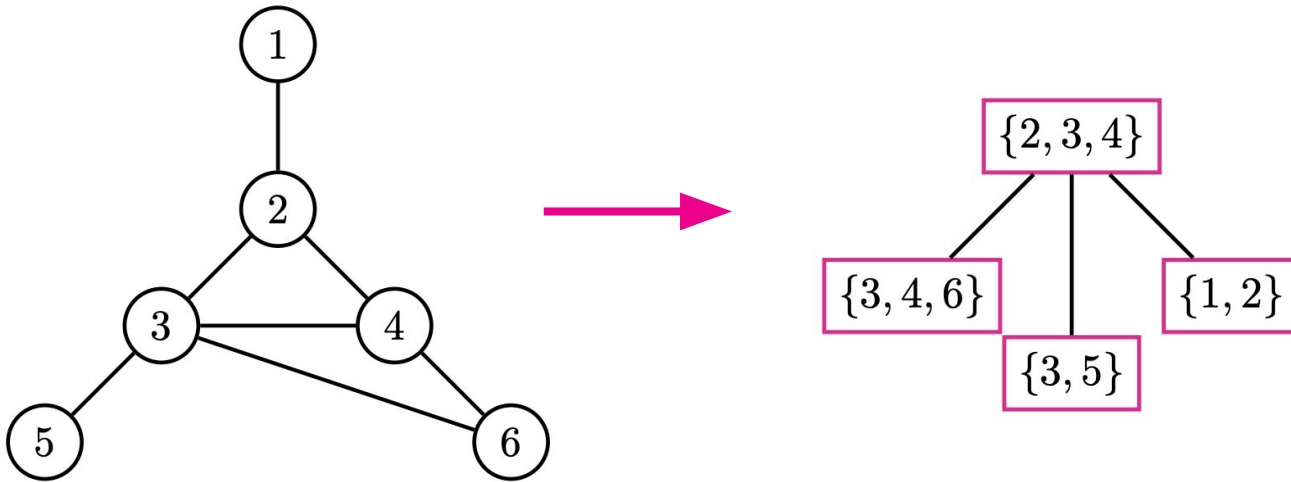
Pathwidth: minimum width of a path decomposition

Treewidth

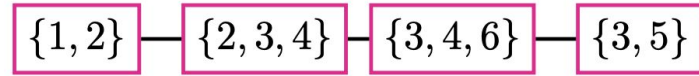
A graph's similarity to a tree

Tree decomposition

Tree of bags of vertices



Treewidth: minimum width of a tree decomposition



This is also a valid tree decomposition

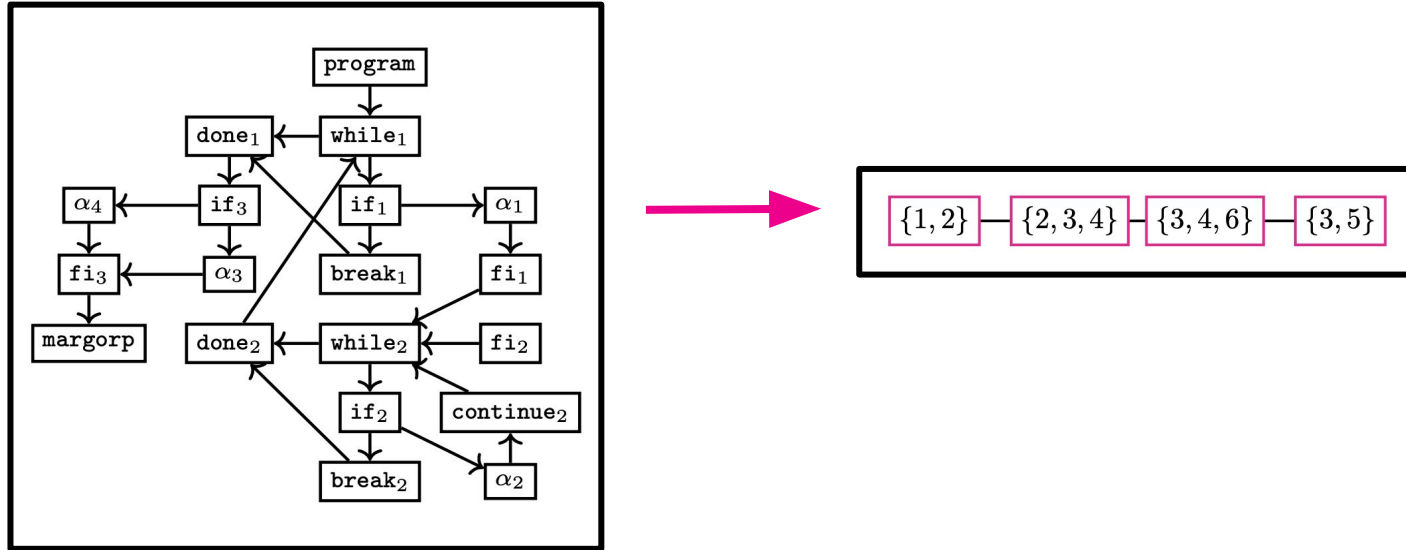
Every path decomposition is also a tree decomposition

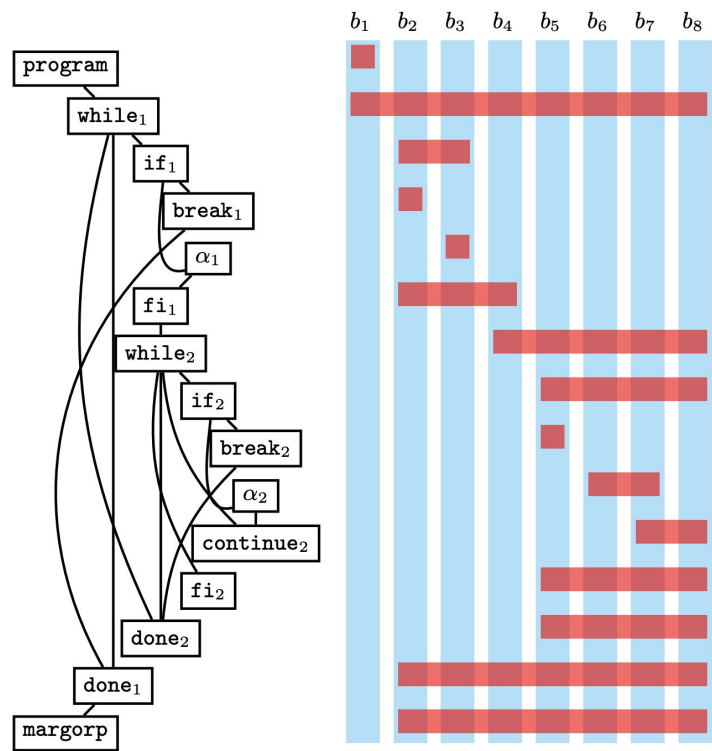
If a graph has a small path decomposition, it also has a small tree decomposition

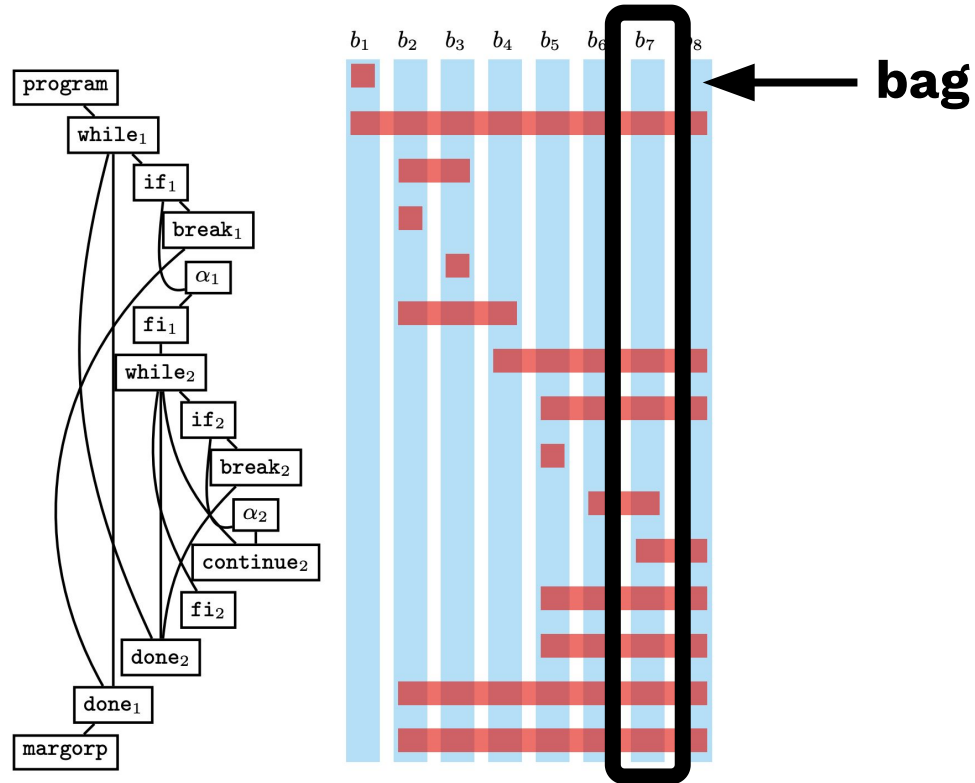
$$\mathbf{tw(G) \leq pw(G)}$$

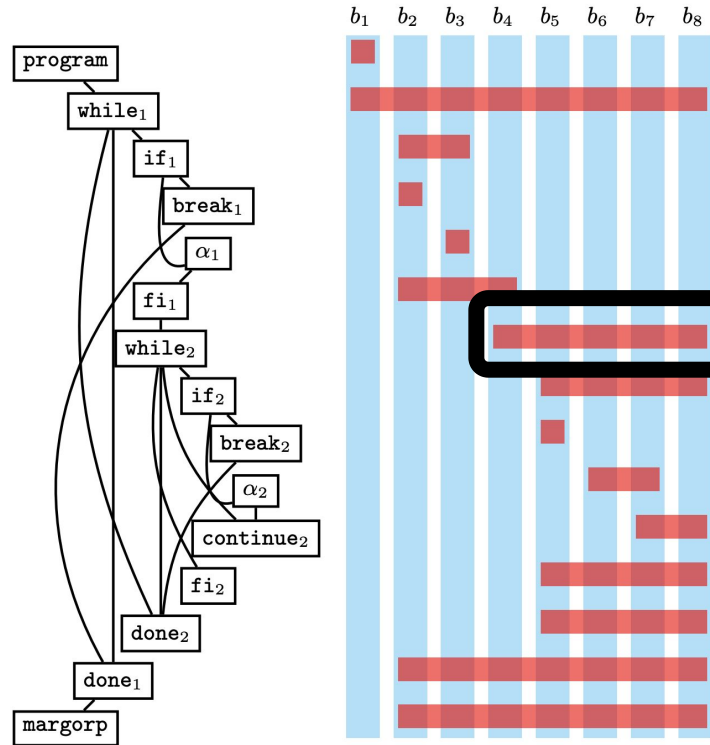
Some problems cannot be hard to solve for graphs of small treewidth, but they can be solved efficiently for graphs of small pathwidth

Pathwidth of control-flow graphs

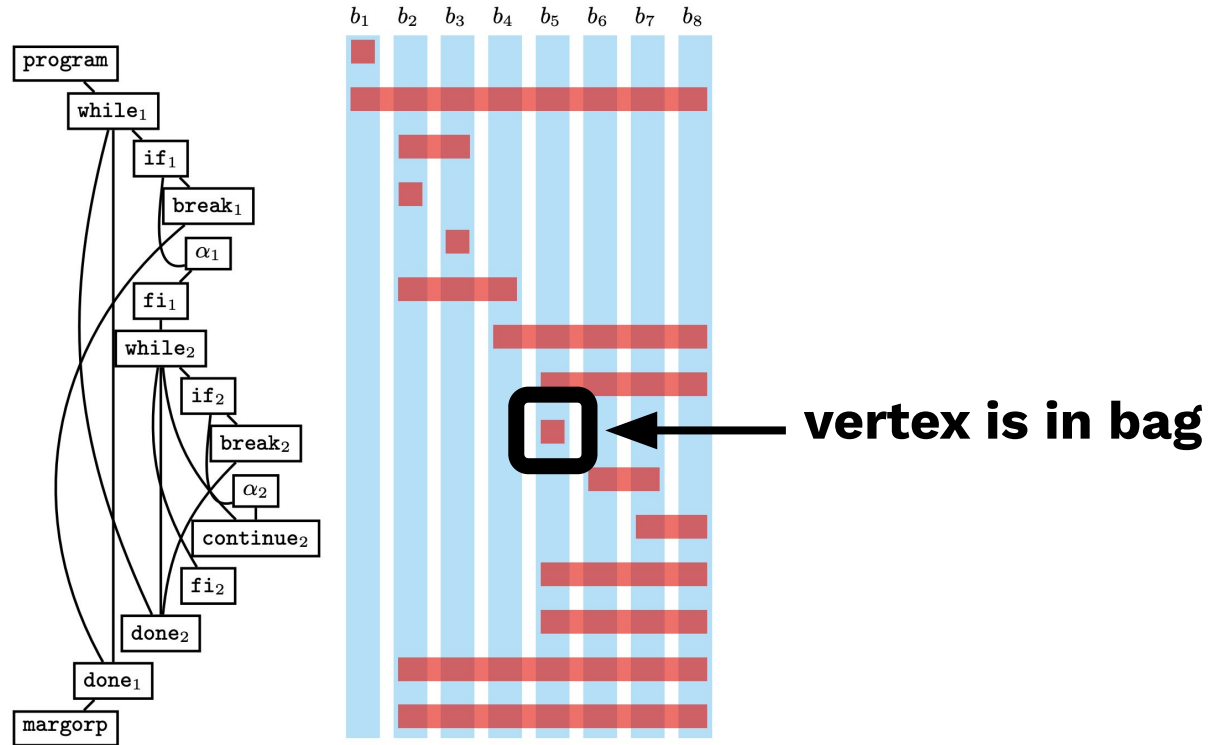


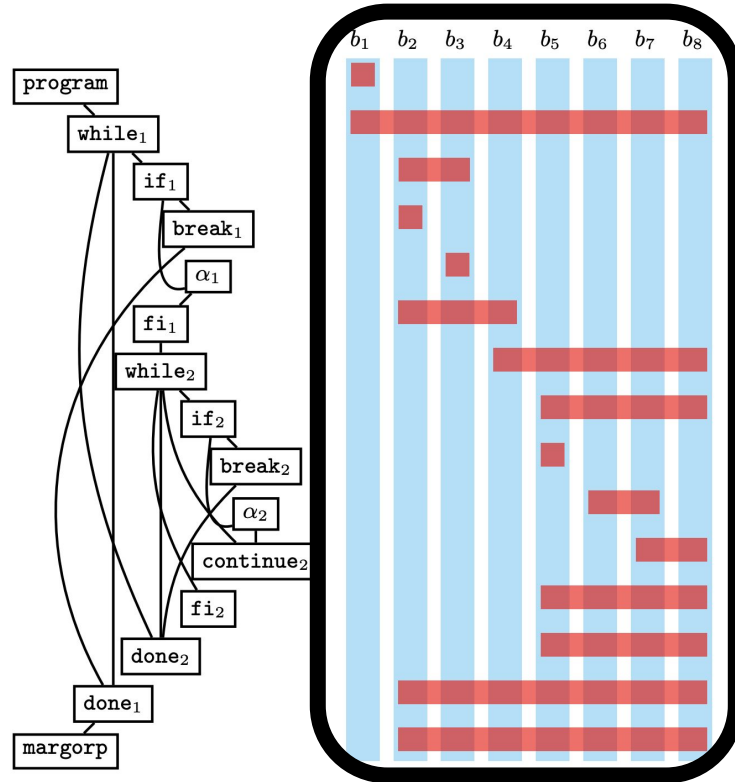




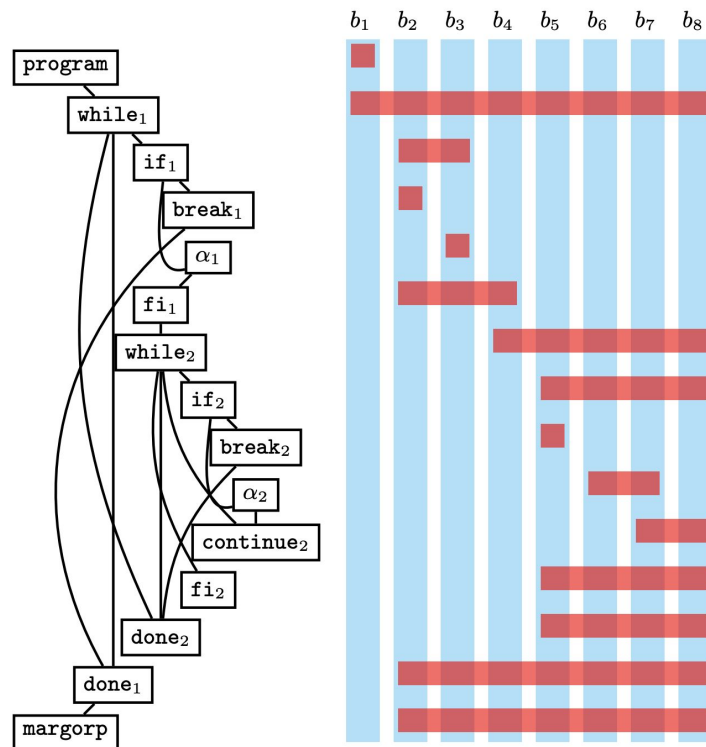


vertex in the
control-flow graph





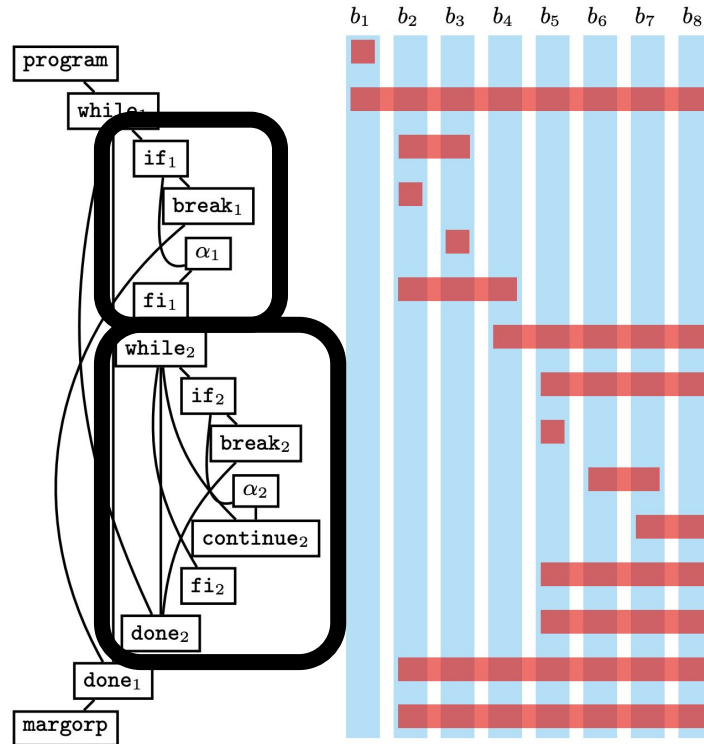
← **width = maximum number of vertices in any bag minus one**



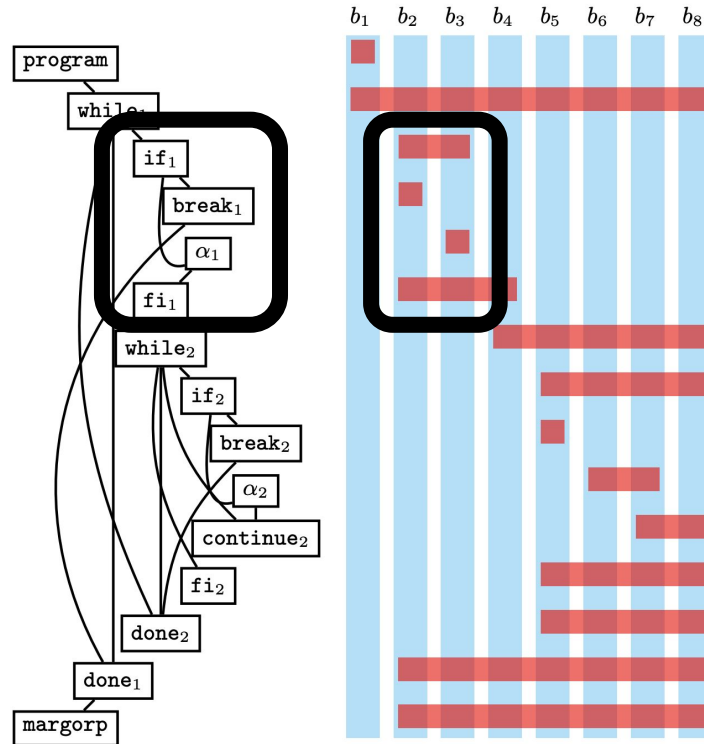
How to generate this decomposition

We need to handle:

- Concatenated statements
- Nested statements

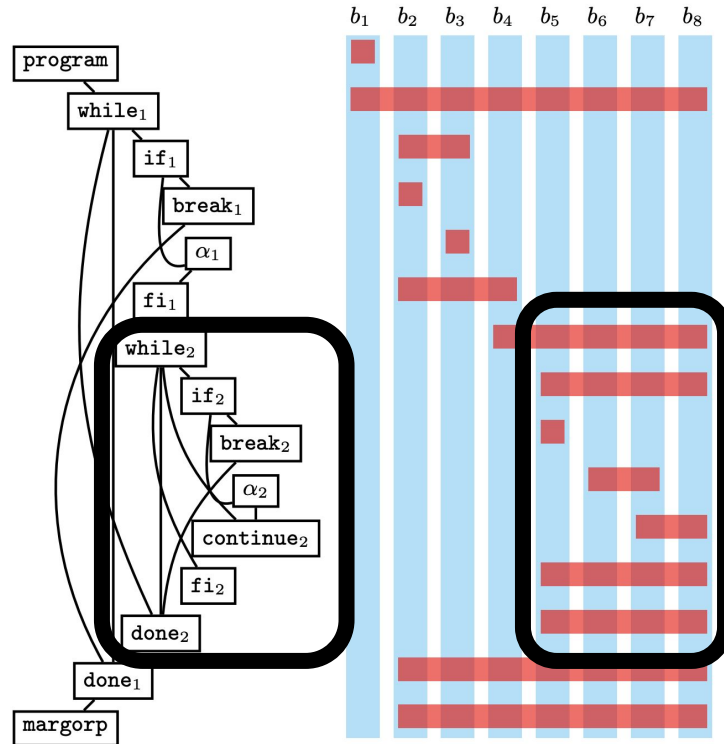


Concatenated statements



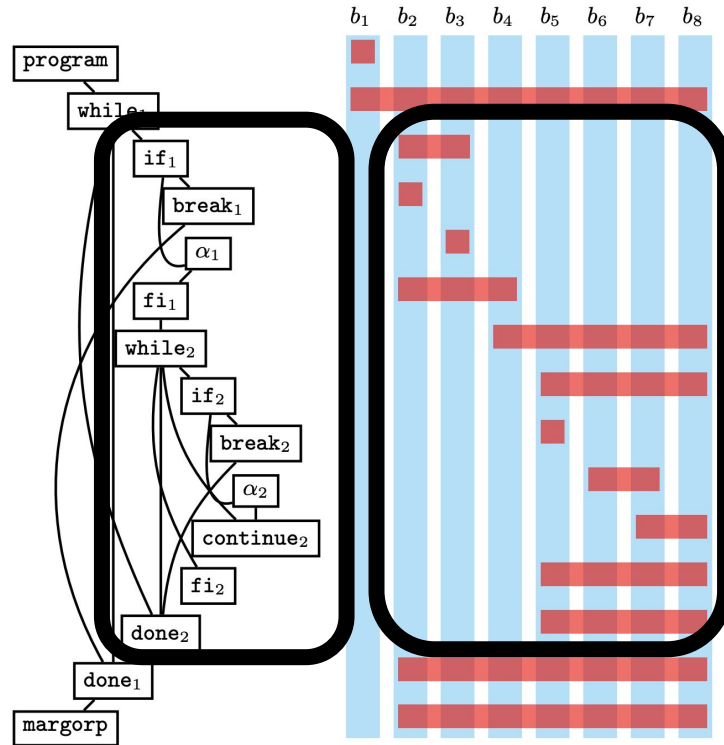
Concatenated statements

- Recursively generate the path decomposition for each statement



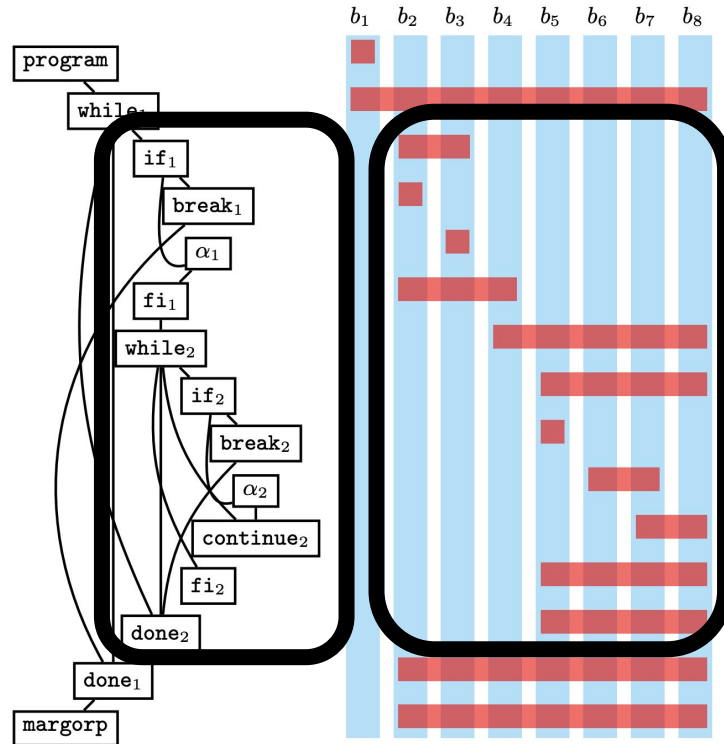
Concatenated statements

- Recursively generate the path decomposition for each statement



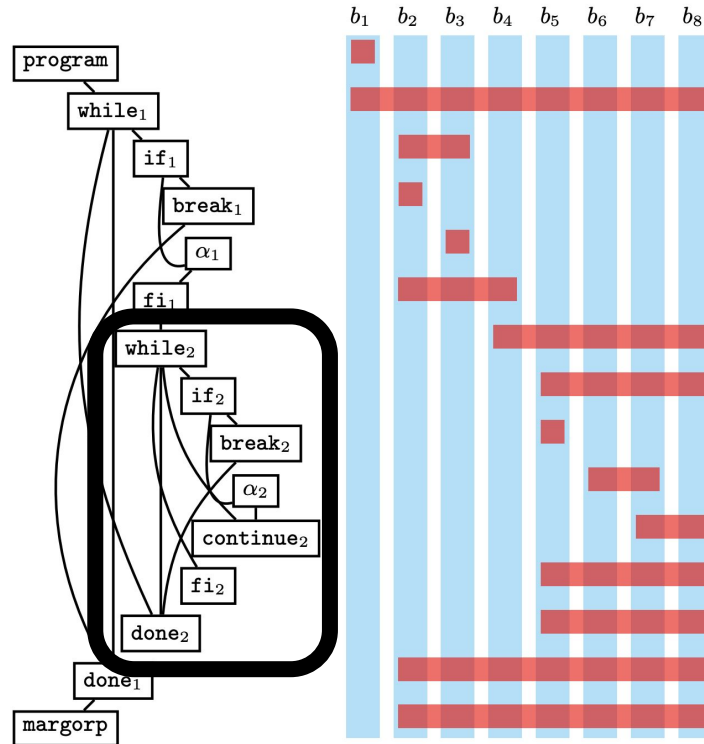
Concatenated statements

- Recursively generate the path decomposition for each statement
- Concatenate them to each other with a small bag in the middle

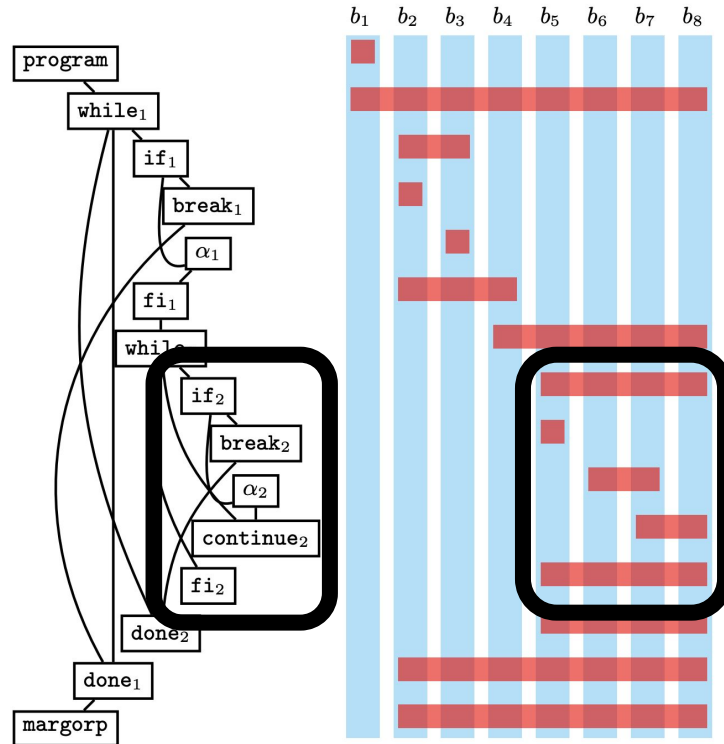


Concatenated statements

- Recursively generate the path decomposition for each statement
- Concatenate them to each other with a small bag in the middle
- Width of decomposition is the maximum between statements

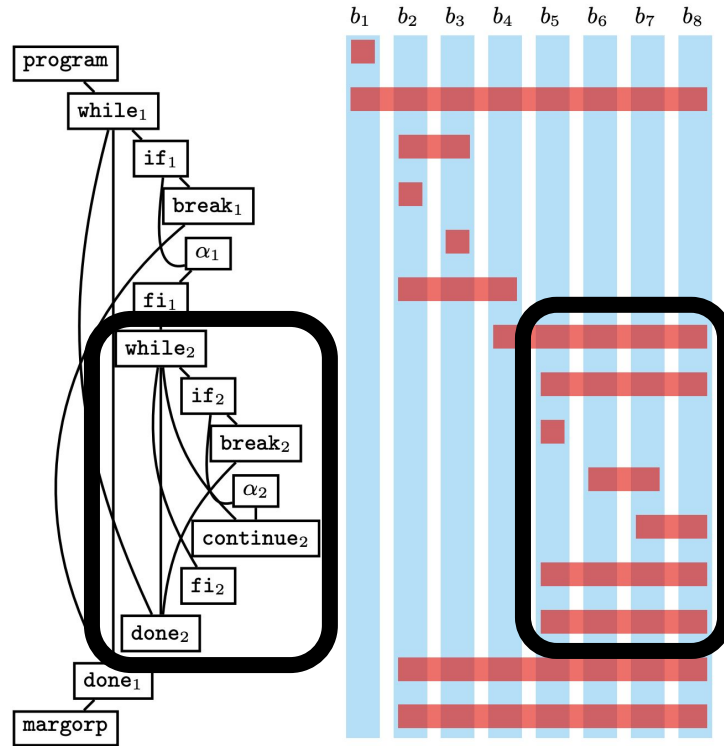


Nested statements



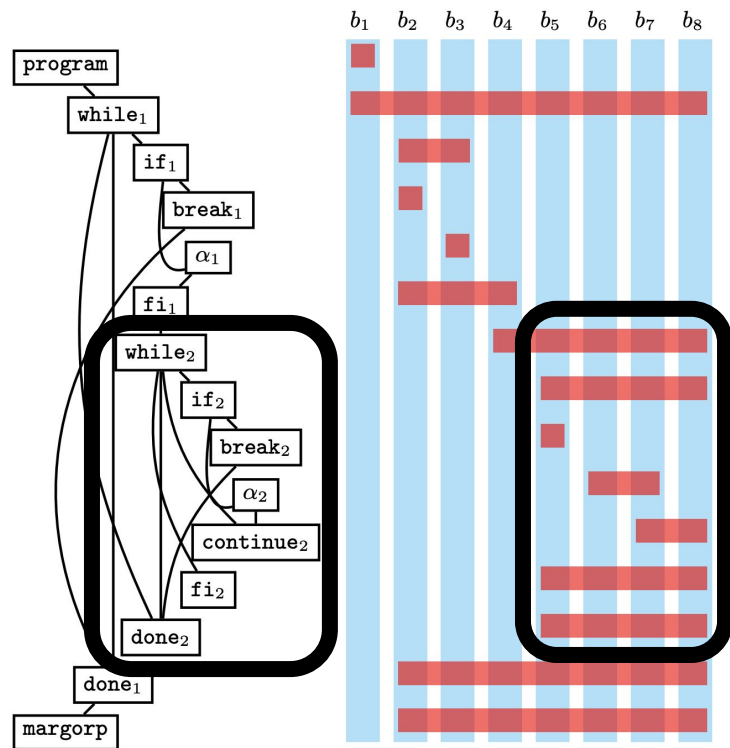
Nested statements

- Recursively generate path decomposition for inner statement



Nested statements

- Recursively generate path decomposition for inner statement
- Add vertices corresponding to the loop to all bags



Nested statements

- Recursively generate path decomposition for inner statement
- Add vertices corresponding to the loop to all bags
- Width of decomposition increases by two

Result

Every time we nest a statement, the width of its decomposition increases by two

The pathwidth of the control-flow graph of a program is at most twice its nesting depth

Goto's can increase the pathwidth

Path decomposition can be obtained directly from program in linear time

Treewidth bound on control-flow graphs

Control-flow graphs of C have a treewidth of at most 7

Goto's can also increase the treewidth

Register Allocation



**Graph coloring on graph of small
pathwidth**



Graph coloring on path decomposition



Dynamic programming

For the register allocation problem

Our algorithm:

$$O(n \cdot w \cdot r^{w \cdot r + r + 1})$$

`n` = number of vertices

`r` = number of registers

`w` = pathwidth

`tw` = treewidth

Treewidth-based algorithm:

$$O(n \cdot r^{2 \cdot tw \cdot r + 2 \cdot r})$$

Experimental results

- We compared our algorithm to the state-of-the-art treewidth-based algorithm
- Since both algorithms are exponential, we need few registers

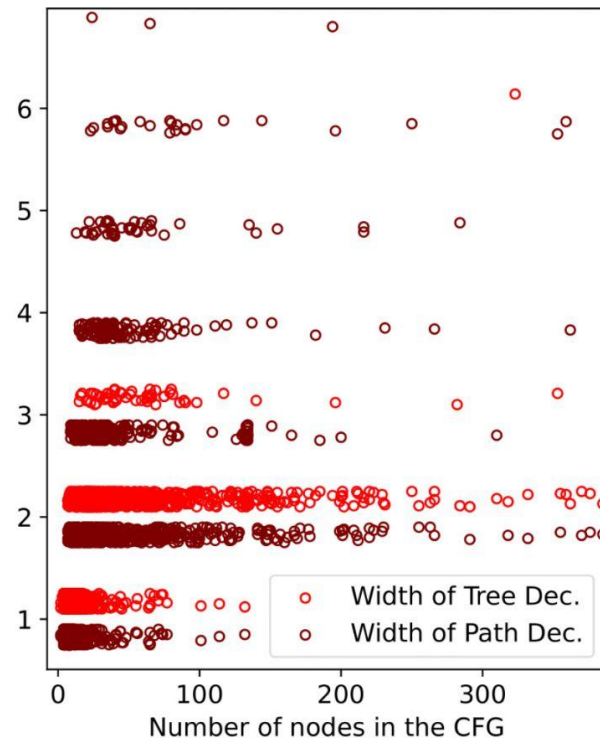
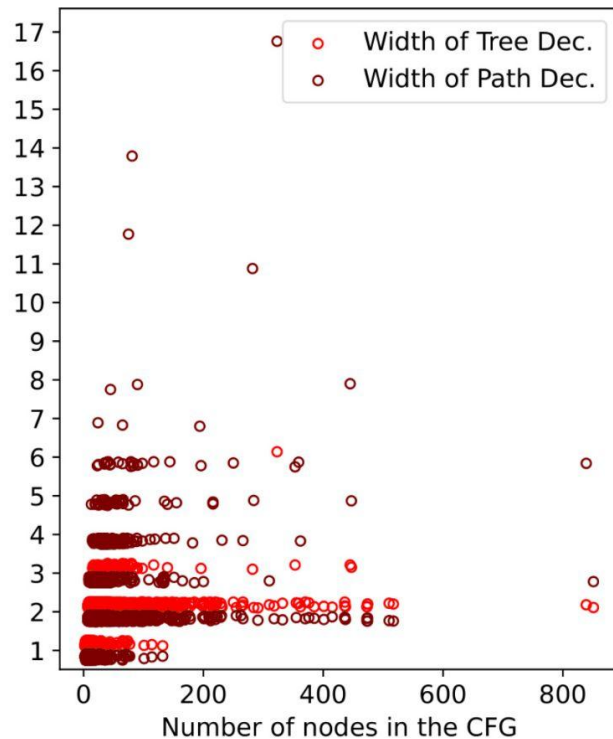
$$O(n \cdot w \cdot r^{w \cdot r + r + 1})$$

$$O(n \cdot r^{2 \cdot tw \cdot r + 2 \cdot r})$$

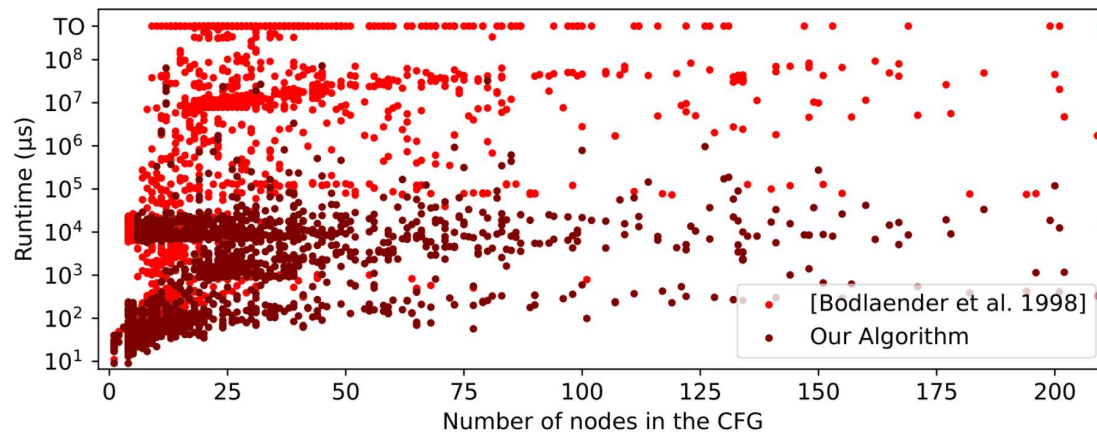
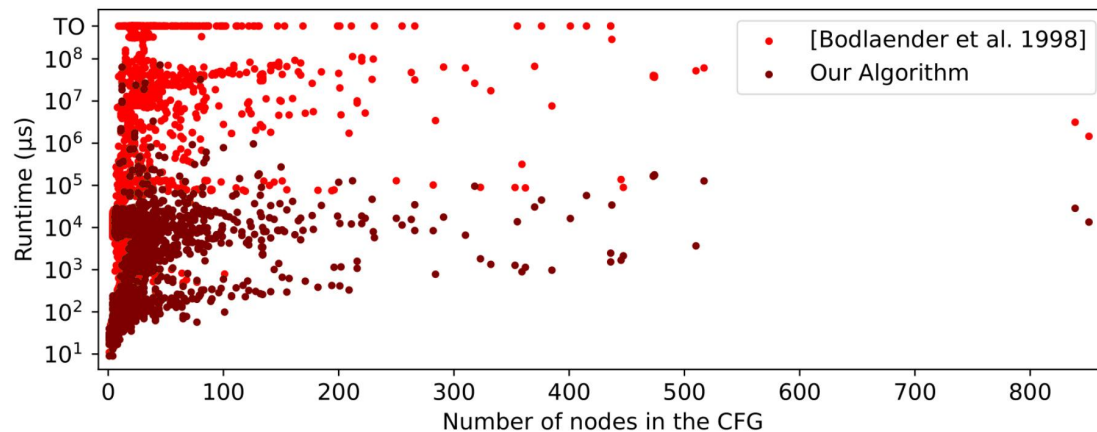
Benchmarks

- Small Device C Compiler test suite, C compiler for 8-bit microcontrollers
- Few registers as is common in embedded systems

Width	Instances	
	TD	PD
1	3276	3276
2	1228	849
3	55	238
4	0	119
5	0	42
6	1	26
7	0	3
8	0	3
11	0	1
12	0	1
14	0	1
17	0	1



Programs may have goto's



Both pathwidth and treewidth are generally fairly small

Even though treewidth is smaller, the pathwidth-based algorithm compensates this through the a better complexity and smaller hidden constants.

Final remarks

Control flow graphs have a pathwidth of at most twice their nesting depth

Using pathwidth as opposed to treewidth is useful because:

- It is a stronger parameter
- All tree-decomposition-based algorithms still work (and may have their complexity improved)
- Hidden constants are better
- Works for labeled break and continue statements
- It is much simpler to design algorithms